PRACTICAL-6

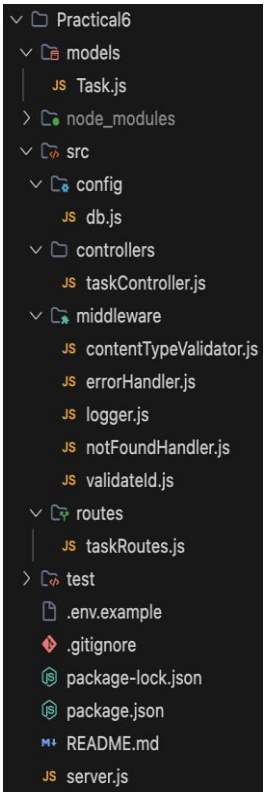
PROBLEM DEFINITION:

• Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.
• Replace the GitHub repo data from Practical 3 with task data from your own backend.
• The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.
• All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.
• Handle loading and error states for every API interaction, not just the initial fetch.

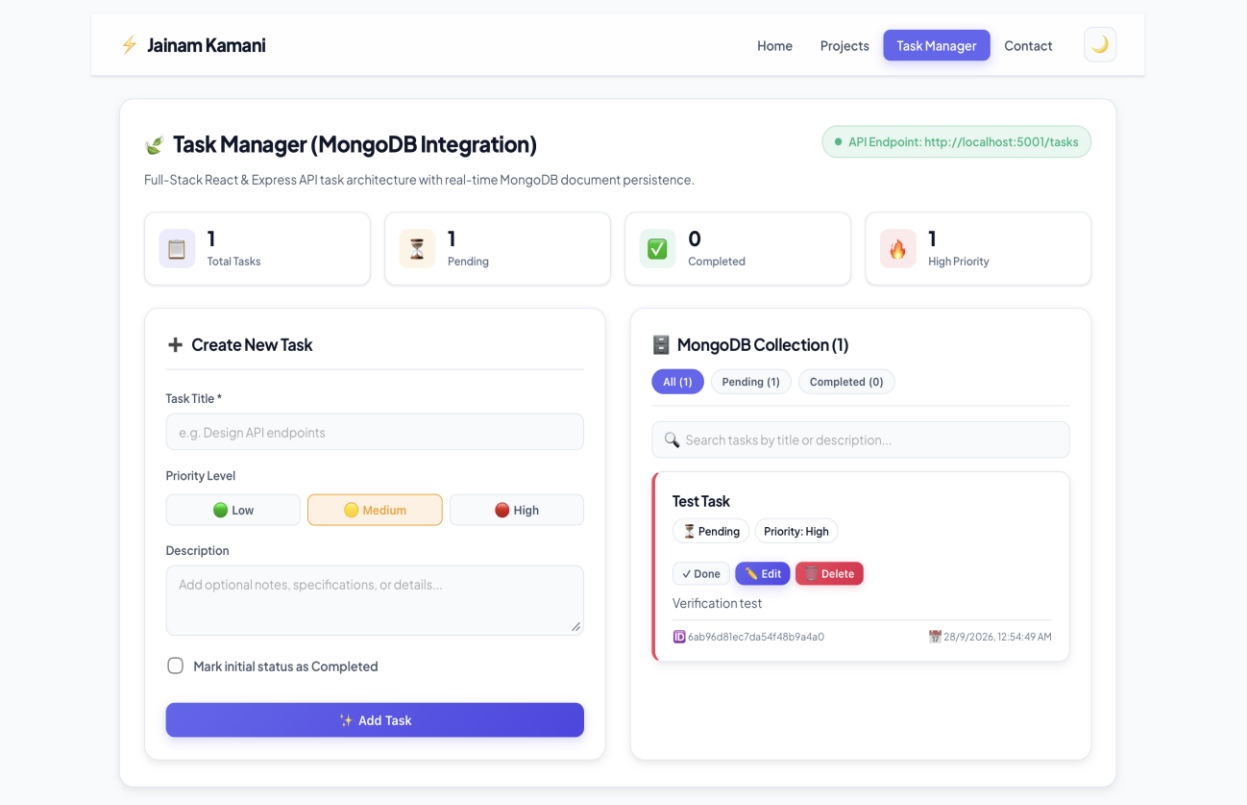
SUPPLEMENTARY PROBLEMS:

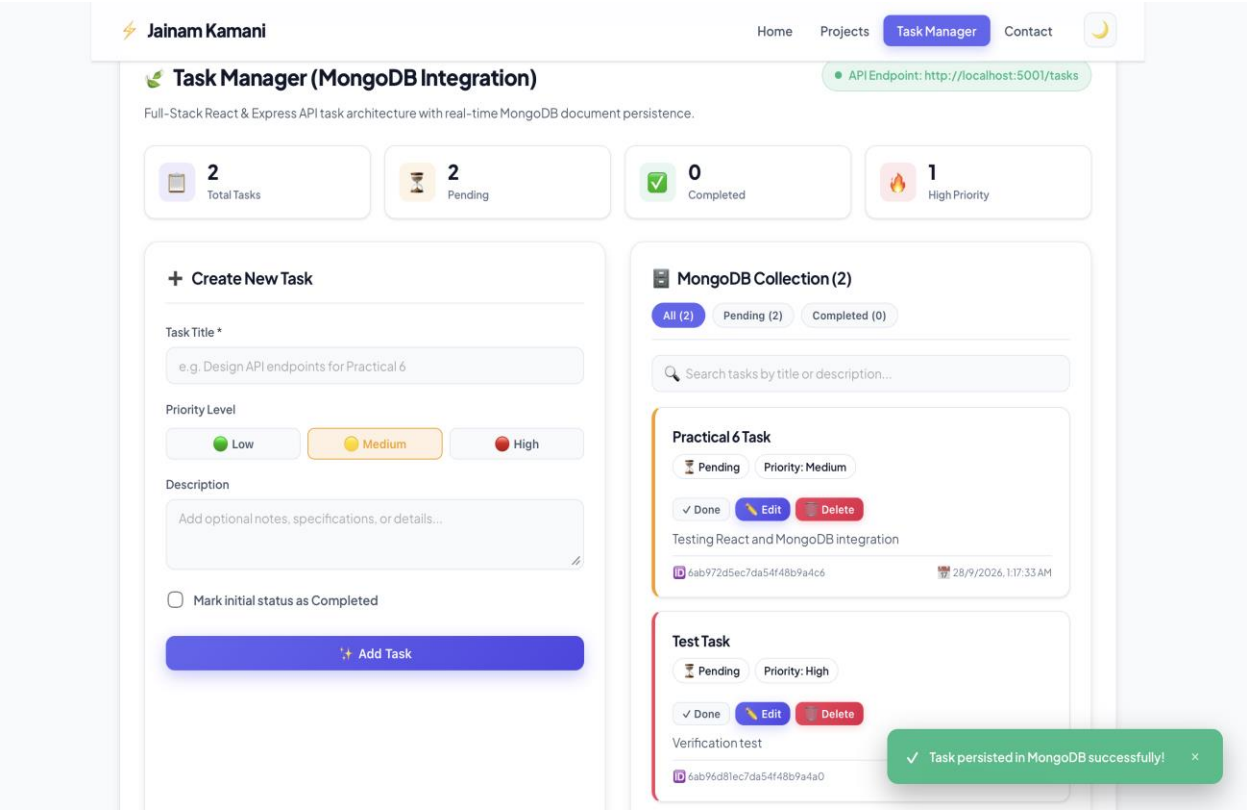
• Add an optimistic UI update for task creation show the new task in the list immediately, before the server confirms.
• Add a confirmation dialog before deleting a task.
• Add a toast/notification component that shows success or failure after each operation.

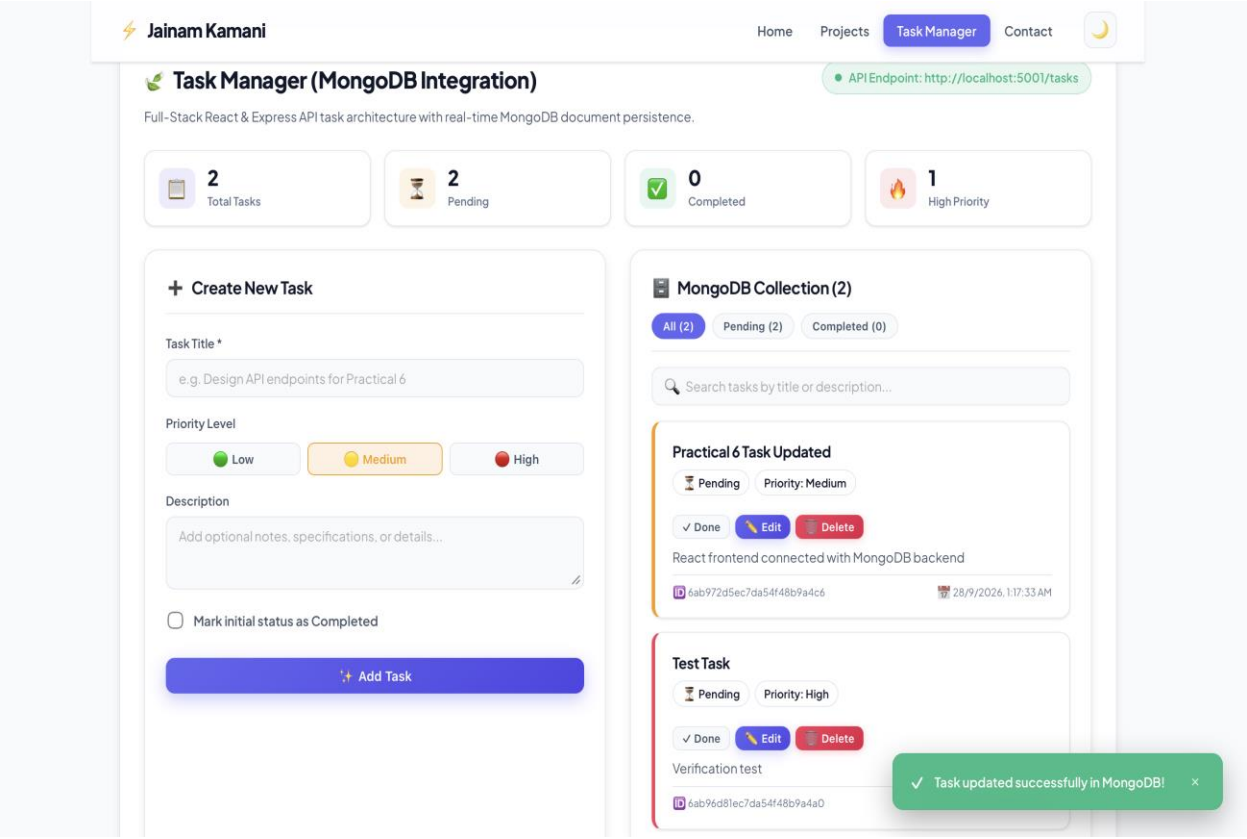
ARCHITECTURE / DIAGRAM:

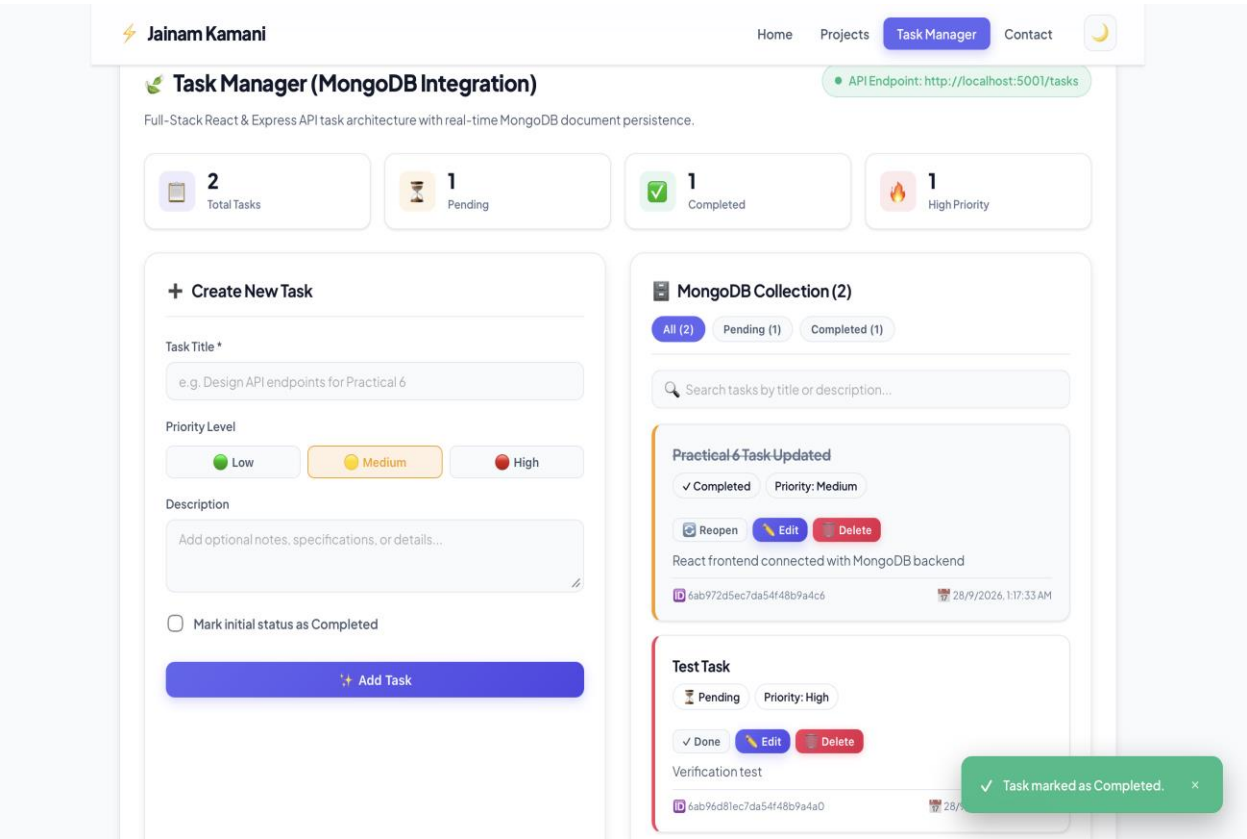


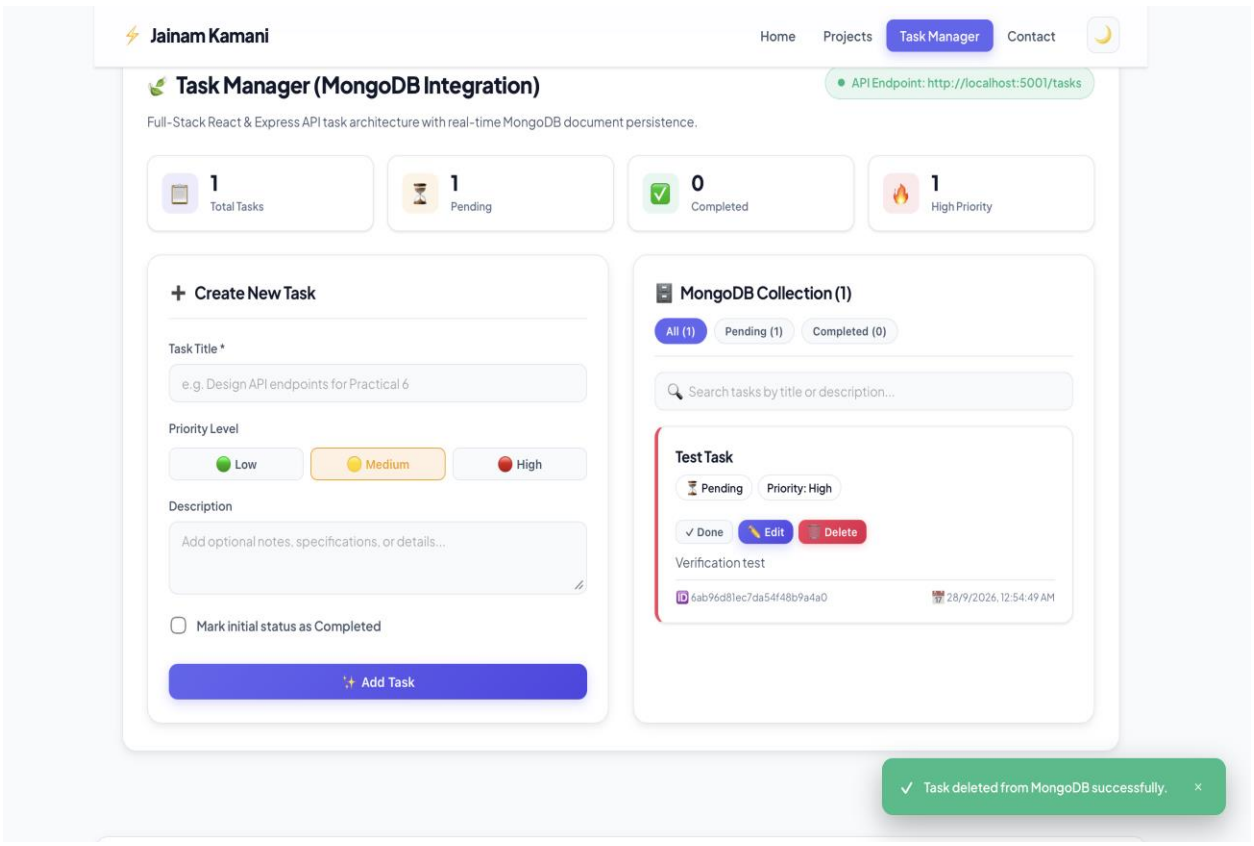
OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

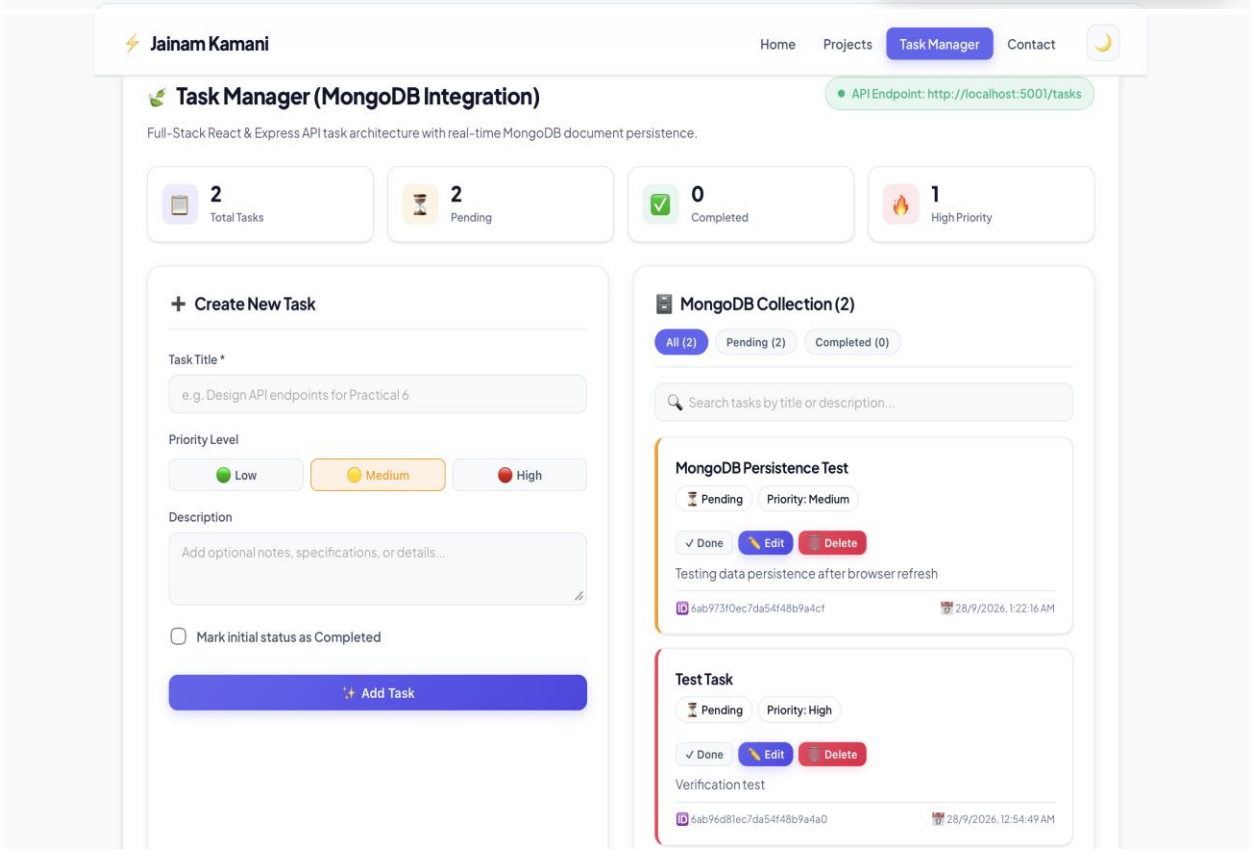





Jainam Kamani
Home
Projects
Task Manager
Contact
Task Manager (MongoDB Integration)
API Endpoint: http://localhost:5001/tasks
Full-Stack React & Express API task architecture with real-time MongoDB document persistence.
2
Total Tasks
2
Pending
0
Completed
1
High Priority
Create New Task
Task Title *
e.g. Design API endpoints for Practical 6
Priority Level
Low
Medium
High
Description
Add optional notes, specifications, or details...
Mark initial status as Completed
Add Task
MongoDB Collection (2)
All (2)
Pending (2)
Completed (0)
Search tasks by title or description...
Practical 6 Task Updated
Pending
Priority: Medium
Done
Edit
Delete
React frontend connected with MongoDB backend
6ab972d5ec7da54f48b9a4c6
28/9/2026, 1:17:33 AM
Test Task
Pending
Priority: High
Done
Edit
Delete
Verification test
6ab96d81ec7da54f48b9a4a0
Task updated successfully in MongoDB!


Jainam Kamani
Home
Projects
Task Manager
Contact
Task Manager (MongoDB Integration)
API Endpoint: http://localhost:5001/tasks
Full-Stack React & Express API task architecture with real-time MongoDB document persistence.
2
Total Tasks
1
Pending
1
Completed
1
High Priority
Create New Task
Task Title *
e.g. Design API endpoints for Practical 6
Priority Level
Low
Medium
High
Description
Add optional notes, specifications, or details...
Mark initial status as Completed
Add Task
MongoDB Collection (2)
All (2)
Pending (1)
Completed (1)
Search tasks by title or description...
Practical 6 Task Updated
Completed
Priority: Medium
Reopen
Edit
Delete
React frontend connected with MongoDB backend
6ab972d5ec7da54f48b9a4c6
28/9/2026, 1:17:33 AM
Test Task
Pending
Priority: High
Done
Edit
Delete
Verification test
6ab96d81ec7da54f48b9a4a0
Task marked as Completed.


Jainam Kamani
Home
Projects
Task Manager
Contact
Task Manager (MongoDB Integration)
API Endpoint: http://localhost:5001/tasks
Full-Stack React & Express API task architecture with real-time MongoDB document persistence.
1
Total Tasks
1
Pending
0
Completed
1
High Priority
Create New Task
Task Title *
e.g. Design API endpoints for Practical 6
Priority Level
Low
Medium
High
Description
Add optional notes, specifications, or details...
Mark initial status as Completed
Add Task
MongoDB Collection (1)
All (1)
Pending (1)
Completed (0)
Search tasks by title or description...
Test Task
Pending
Priority: High
Done
Edit
Delete
Verification test
6ab96d81ec7da54f48b9a4a0
28/9/2026, 12:54:49 AM
Task deleted from MongoDB successfully.


Jainam Kamani
Home
Projects
Task Manager
Contact
Task Manager (MongoDB Integration)
API Endpoint: http://localhost:5001/tasks
Full-Stack React & Express API task architecture with real-time MongoDB document persistence.
2
Total Tasks
2
Pending
0
Completed
1
High Priority
Create New Task
Task Title *
e.g. Design API endpoints for Practical 6
Priority Level
Low
Medium
High
Description
Add optional notes, specifications, or details...
Mark initial status as Completed
Add Task
MongoDB Collection (2)
All (2)
Pending (2)
Completed (0)
Search tasks by title or description...
MongoDB Persistence Test
Pending
Priority: Medium
Done
Edit
Delete
Testing data persistence after browser refresh
6ab973f0ec7da54f48b9a4cf
28/9/2026, 1:22:16 AM
Test Task
Pending
Priority: High
Done
Edit
Delete
Verification test
6ab96d81ec7da54f48b9a4a0
28/9/2026, 12:54:49 AM

KEY QUESTIONS / ANALYSIS:

1) What changes are required on the backend (CORS) to allow the React dev server to call the Express API?

We need to install and configure the cors middleware in Express. We must allow the React frontend origin, http://localhost:5173, to access the backend API running at http://localhost:5000. This enables cross-origin HTTP requests between the frontend and backend.

2. Why should the UI re-fetch or update local state after a successful POST/PUT/DELETE rather than assuming success silently?

The UI should re-fetch data or update its local state after a successful POST, PUT, or DELETE request to reflect the latest database changes. This keeps the task list synchronized with MongoDB and provides immediate feedback to the user.

3. What is the risk of not handling errors on write operations (POST/PUT/DELETE) the same way as read operations (GET)?

If errors in write operations are not handled properly, users may believe a task was created, updated, or deleted when the operation actually failed. This can cause inconsistent UI data, confusion, and loss of trust. Proper error handling allows the application to display meaningful messages and maintain data consistency.

GITHUB REPOSITORY:

https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks